

DevNexus | AgileBridge



UNITRADE



SRS Document

Prepared By :

DevNexus

devnexus28@gmail.com

1. Introduction

1.1 Overview

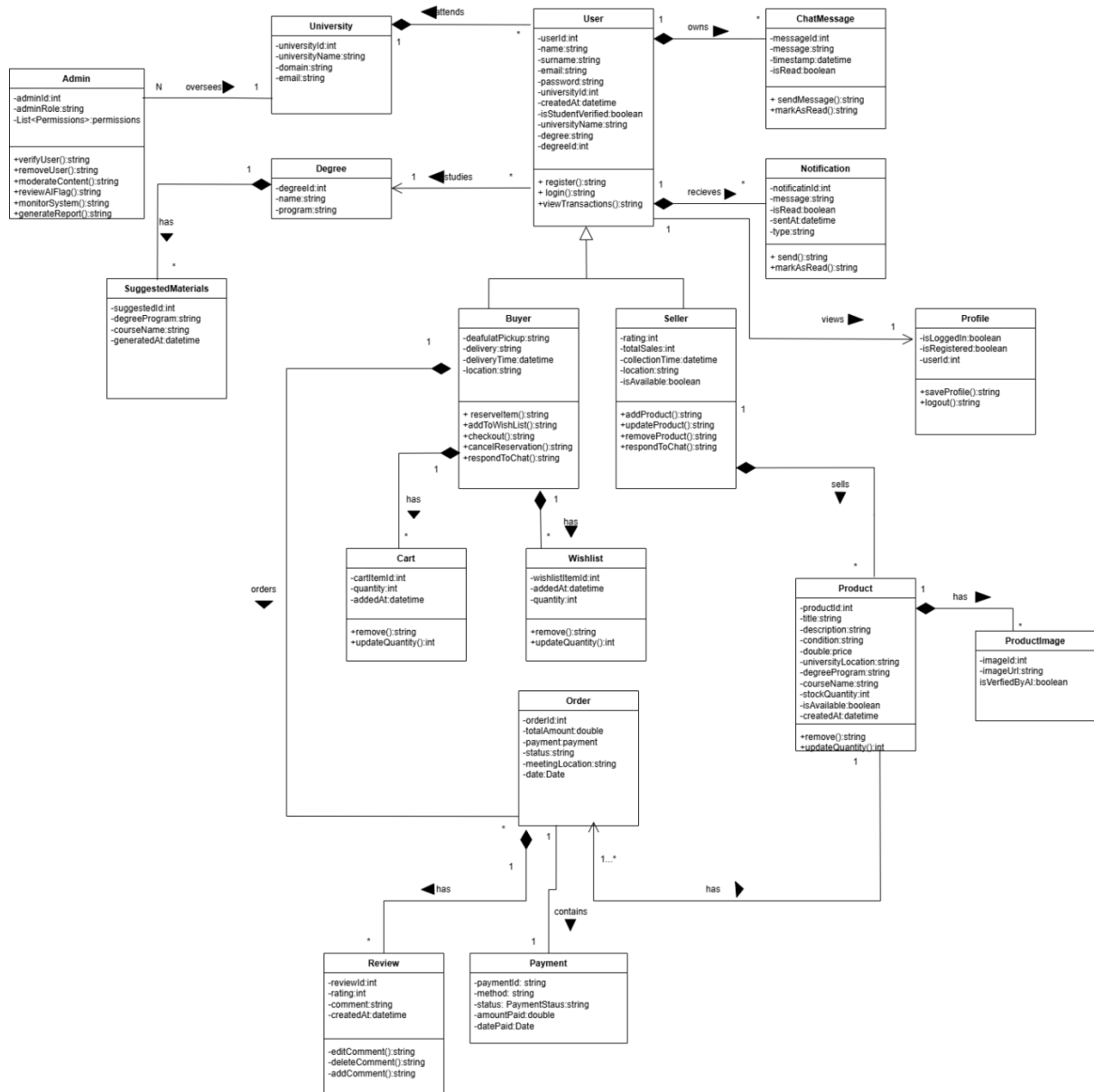
UniTrade is a web and mobile platform designed to connect university students who want to buy and sell second-hand academic materials. Built for the South African university context, where students face significant financial pressure from high tuition fees and expensive prescribed textbooks, the system provides a safe, verified, and cost-effective marketplace for books and study resources.

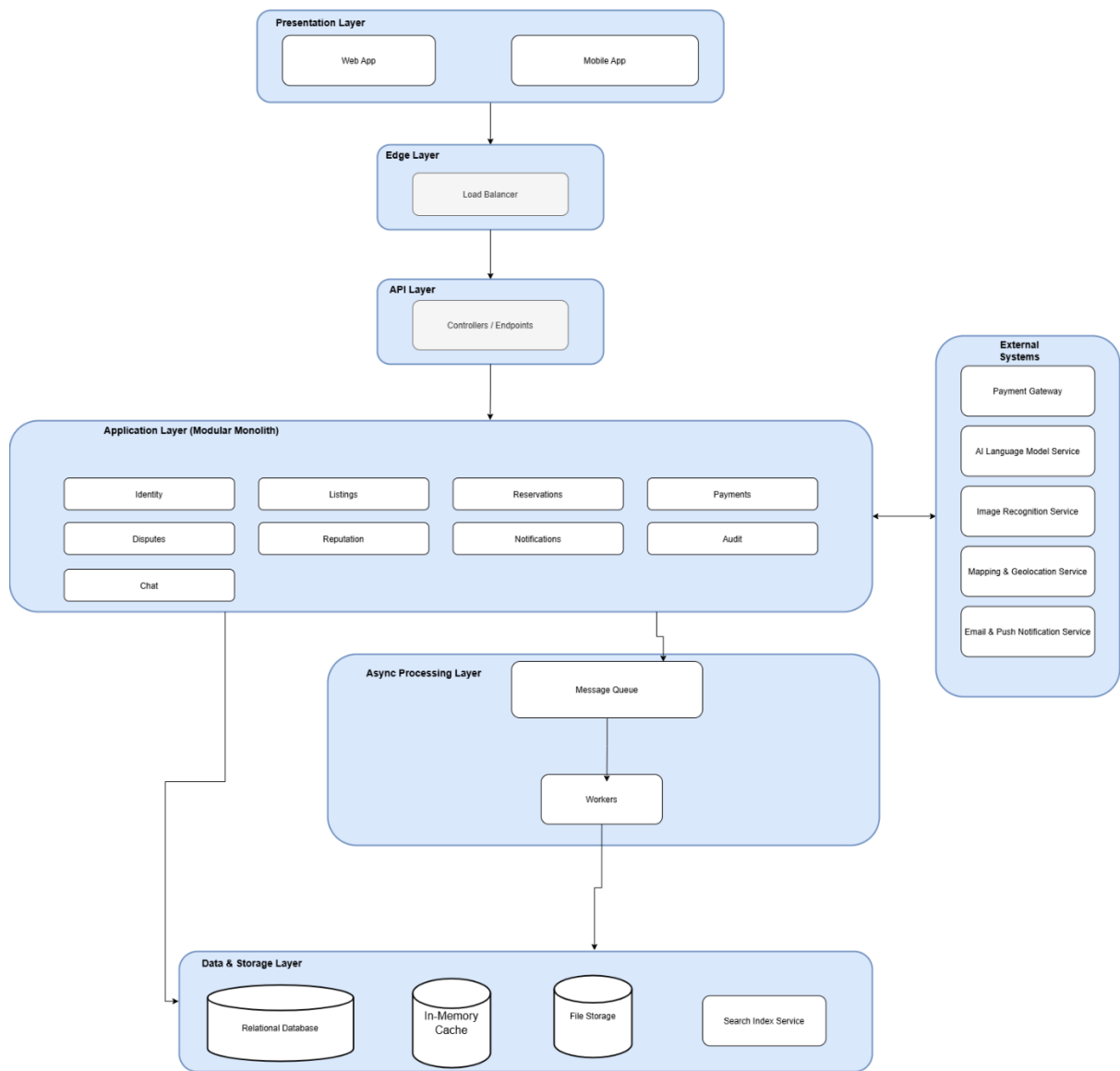
Currently, students either purchase new materials at full price or rely on informal networks to find second-hand options, which is unreliable and often unsafe. UniTrade solves this by introducing a structured platform where students can list, browse, and purchase used materials directly from verified peers.

1.2 Key System Features

- AI-assisted listing verification
- OTP-based student authentication
- Real-time chat between buyers and sellers
- GPS meetup coordination
- OZOW payment processing
- POPIA-compliant data handling

1.3 Domain Model





2. Functional Requirements

R1: User and Identity Management

R1.1 User Registration

- **R1.1.1:** The system shall allow a registered university student to be registered in the system.
 - **R1.1.2:** The system shall provide an input area for student email, student number and proof of registration PDF.
 - **R1.1.3:** The system shall verify that the user is a student at said university by sending an OTP to the user's university email.
 - **R1.1.4:** The system shall allow users to resend OTP to their email if it expires.
 - **R1.1.5:** The system shall extract text from uploaded proof of registration PDF and compare the student number and degree against the registration form data using AI.
 - **R1.1.6:** The system shall assign a confidence score (0–100) to the AI verification result.
 - **R1.1.7:** The system shall provide manual verification of students' degrees if AI fails or gives a score below 75.
 - **R1.1.8:** The system must be able to provide users with a verification status.
 - **R1.1.9:** The system shall allow users to resubmit proof of registration PDF if manual verification fails.
-
- **R1.2:** The system shall allow users to remain logged into the website or mobile application.
 - **R1.3:** The system shall allow logged in users to reset their password.
 - **R1.4:** The system shall allow users to switch roles between being a seller or buyer.
 - **R1.5:** The system shall allow logged in users to log out.
 - **R1.6:** The system shall restrict unverified users to read-only access.
 - **R1.7:** The system shall prevent unverified users from initiating chats.
 - **R1.8:** The system shall prevent unverified users from making purchases.
 - **R1.9:** The system shall store listings from unverified sellers as drafts only.

R2: Listing and Inventory Management

R2.1 Create Listing Draft

- **R2.1.1:** The system shall allow users to upload images of materials.
- **R2.1.2:** The system shall allow users to provide details of the product they are selling.
- **R2.1.3:** The system shall use AI to assess uploaded images for object recognition (must identify as a book or educational material) before it is published.
- **R2.1.4:** The system shall assign a quality score (0–100) to each image.
- **R2.1.5:** The system shall automatically approve users when AI confidence score exceeds threshold.

- **R2.1.6:** The system shall have the admin intervene to manually check quality if score is below threshold.
- **R2.1.7:** The system shall classify listings into low, medium, and high risk categories.
- **R2.1.8:** The system shall apply different visibility rules based on risk level.
- **R2.2:** The system shall allow sellers to edit their listings.
- **R2.3:** The system shall allow sellers to remove their listings.
- **R2.4:** The system shall allow sellers to view all their listings.
- **R2.5:** The system shall allow sellers to save their listing so it's in the process of getting published.
- **R2.6:** The system shall keep all new listings in a draft state until submission.
- **R2.7:** The system shall prevent unverified sellers from submitting listings.
- **R2.8:** The system shall publish listings automatically for low-risk scores.
- **R2.9:** The system shall reduce visibility of medium-risk listings.
- **R2.10:** The system shall prevent high-risk listings from being published.
- **R2.11:** The system shall allow sellers to unreserve the listing the buyer has reserved on their material.

R3: Discovery and Recommendations

R3.1 Degree-Specific Dashboard

- **R3.1.1:** The system shall provide users with suggested materials.

R3.2 Search

- **R3.2.1:** Buyers can search by seller name.
- **R3.2.2:** Buyers can search listings by material title.
- **R3.2.3:** Buyers can search listings by course code.

R3.3 Filters

- **R3.3.1:** Buyers shall be able to filter search results by price range.
- **R3.3.2:** Buyers shall be able to filter search results by date listed.
- **R3.3.3:** The system shall allow buyers to view details of the material.
- **R3.4:** The system shall allow buyers to reserve listings.
 - **R3.4.1:** Buyers must be able to proceed to checkout.
- **R3.5:** The system shall allow buyers to add material to wishlist.

R4: Reservation and Transaction

R4.1 Reservation Rules

- **R4.1.1:** The system shall allow buyers to reserve items for a 24-hour window period.
- **R4.1.2:** The system shall require seller acknowledgement within 24 hours.

- **R4.1.2b:** The system shall allow buyers to reserve a full degree list of materials.
- **R4.1.3:** The system shall allow buyers to reserve the full degree package for days.
- **R4.1.4:** The system shall allow sellers to manually release reservations after 12 hours of buyer inactivity.
- **R4.1.5:** The system shall pause reservation timers when both users are actively communicating.
- **R4.1.5b:** The system shall provide auto cancellation for reservations that surpass the given time frame.
- **R4.1.6:** The system shall expire reservations if either party is inactive.
- **R4.2:** The system shall provide a payment gateway to process buyers' payments.
- **R4.3:** The system shall allow buyers to confirm payment once they've met up with the seller.
 - **R4.3.1:** The system shall generate a pin for the buyer.
 - **R4.3.2:** The system shall allow sellers to enter a pin given by the buyer during their meet-up.
- **R4.4:** The system shall allow buyers to raise a dispute if materials were not as they were displayed.

R5: Admin and Dispute Resolution

R5.1 Verification Management

- **R5.1.1:** Admin must be able to approve/reject proof of enrollment.

R5.2 Listing Review

- **R5.2.1:** Admins must be able to manually reject/approve materials.

R5.3 Dispute Handling

- **R5.3.1:** Admins must be able to review evidence.
- **R5.3.2:** Admins must be able to suspend malicious users.
- **R5.3.3:** Admins must be able to flag users as a warning.
- **R5.4:** The system shall provide an admin interface to submit and store POPIA compliance documentation.

R6: Communication and Logistics

R6.1 Real-Time Chat

- **R6.1.1:** Users must be able to open a chat thread.
 - **R6.1.1.1:** Users must be able to send a message.
 - **R6.1.1.2:** Users must be able to get a notification from other users.

R6.2 GPS Meetup

- **R6.2.1:** Users must be able to select a meeting point from the provided map interface (Google Maps).
- **R6.2.2:** The system shall allow either party to propose a meeting point; the other party must accept before it is confirmed.
- **R6.3:** The system shall allow users to add a review once a transaction has been completed.

3. Quality Requirements

QR1: Performance

- **QR1.1:** The system needs to return product search results within 2 seconds under normal load (approximately 100 concurrent users).
- **QR1.2:** The system needs to ensure that page loading times and product listings are less than 3 seconds on a 4G network.
- **QR1.3:** Checkout procedures (payment confirmation) need to be completed within 5 seconds in 95% of cases.
- **QR1.4:** Real-time chat messages shall be delivered within 500 milliseconds under normal network conditions.
- **QR1.5:** Push notifications shall be delivered within 3 seconds of the triggering event.

QR2: Reliability

- **QR2.1:** The system needs to be available 99% of the time on a monthly basis.
- **QR2.2:** The system needs to recover from server failure within 60 seconds using auto restart or failover mechanisms.
- **QR2.3:** Data loss during a crash needs to be limited to less than 1 transaction lost via regular backups and atomic operations.
- **QR2.4:** In the event of AI verification service failure, the system shall automatically fall back to manual admin review without disrupting user access to other features.

QR3: Scalability

- **QR3.1:** The system shall support a minimum of 500 concurrent users during peak periods (e.g. semester start) without degradation in response time.
- **QR3.2:** The system shall be architected to scale horizontally on Azure App Services to accommodate growth in user base without requiring significant re-architecture.
- **QR3.3:** The AI verification service shall be deployed as an independent service to allow it to scale separately from the core application during periods of high listing activity.

QR4: Security

- **QR4.1:** All users must be authenticated via a university email OTP mechanism before gaining any access to the platform.
- **QR4.2:** All sensitive data including personal information and payment details shall be encrypted.
- **QR4.3:** The system shall comply fully with the Protection of Personal Information Act (POPIA) of South Africa, including data minimisation, user consent, and the right to access and delete personal data.
- **QR4.4:** All payment transactions shall be processed through OZOW. The UniTrade application shall not store raw card or banking details at any point.

- **QR4.5:** The system shall implement role-based access control (RBAC) to ensure buyers, sellers, and administrators can only access functionality appropriate to their role.

QR5: Usability

- **QR5.1:** A new user shall be able to complete registration, browse listings, and add an item to cart within 5 minutes without requiring any training or documentation.
- **QR5.2:** The application shall conform to WCAG 2.1 Level AA accessibility standards, including minimum colour contrast ratio of 4.5:1, descriptive alt text, labelled form fields, and actionable error messages.
- **QR5.3:** The web interface shall be fully responsive across desktop, tablet, and mobile screen sizes.
- **QR5.4:** The mobile application shall support both iOS and Android platforms.
- **QR5.5:** The GPS meetup feature shall provide a text-based address alternative for users who have difficulty interacting with map interfaces.

QR6: Maintainability

- **QR6.1:** All source code shall follow agreed team coding standards and be reviewed via pull requests before merging into main branches.
- **QR6.2:** Automated CI/CD pipelines shall be configured via GitHub Actions or Azure DevOps to deploy to development and UAT environments on every approved merge.
- **QR6.3:** The system shall implement centralised logging and error monitoring using Azure Monitor to enable rapid fault diagnosis.

QR7: Administration Requirements

- **QR7.1:** An administrator shall complete the manual verification of a student's proof of registration within 2 business days of the verification request being submitted.
- **QR7.2:** The system shall send the student an automated notification if their verification has not been processed within 1 business day, informing them of the expected resolution time.
- **QR7.3:** An administrator shall resolve a buyer-seller dispute within 5 business days of the dispute being raised.
- **QR7.4:** The system shall automatically notify both parties involved in a dispute of the admin's final decision within 24 hours of the decision being made.
- **QR7.5:** The system shall automatically freeze any funds related to a dispute immediately upon the dispute being raised, pending admin resolution.
- **QR7.6:** If a buyer does not confirm receipt or raise a dispute within 48 hours of the agreed meetup time, the system shall automatically release funds to the seller and close the transaction.
- **QR7.7:** The admin dashboard shall display all pending verifications and active disputes sorted by submission date to ensure oldest cases are prioritised first.

QR8: Legal and Regulatory Compliance

- **QR8.1:** The system shall comply with POPIA for all personal data handling.
- **QR8.2:** The system shall include clear Terms of Service and Privacy Policy accessible to all users before registration.
- **QR8.3:** The system shall maintain an audit trail of all transactions, disputes, and admin decisions for a minimum period of 3 years in compliance with South African financial record-keeping requirements.

Architectural Patterns

1) Client Server Architecture

UniTrade is built on a client-server architecture. There are two thin clients — a React web app and a React Native mobile app — and they only handle presentation and user interaction. All the important stuff like business logic, data storage, verification, payments, and moderation lives on the server.

The clients don't talk to each other directly and can't bypass the server for anything.

UniTrade does support peer-to-peer transactions in the real world (buyers and sellers meet physically to exchange goods). But the platform itself is still strictly client-server. The server is the trusted middle layer for reservation handling, payment confirmation, PIN verification, and dispute management. There is no peer-to-peer

networking at the system level.

This design choice is intentional. A peer-to-peer architecture would weaken trust and safety guarantees, especially around verified identity, audit trails, and dispute handling. Centralised control is required for the verification and moderation model UniTrade depends on.

2) Communication Patterns and Protocols

- Request-Response (Primary Pattern)
 - Most interactions use a synchronous request-response flow. Anything the user does — browsing listings, registering, expressing interest, or making payments — follows this pattern. The client sends a request and waits for the server's response before continuing.
 - The system uses REST over HTTP/HTTPS.
 - REST was chosen because the API is naturally resource-based (users, listings, reservations, transactions), and it maps cleanly to REST concepts. It also works easily across both React web and React Native without extra tooling.
 - So, REST gives the best balance of simplicity and compatibility for the current stage.
- Message Brokering (Asynchronous Pattern)
 - Some operations don't need to block the user request and are better handled in the background. These include things like reservation expiry timers, notifications, and AI-based processing for listings or documents.
 - These run through Azure Service Bus using an asynchronous messaging pattern.
 - Example: when a reservation is created, the system publishes a message to start a 24-hour expiry timer and immediately returns a response to the user. A background worker then handles the timing logic separately.
 - This decouples slow or delayed operations from user-facing flows.
 - The messaging protocol is AMQP (Advanced Message Queuing Protocol), which supports reliable delivery, ordering, and acknowledgements. That's important for things like reservation expiry or payment notifications where message loss would cause real problems.

- Push Model (Real-Time Communication)
 - Chat and live notifications need instant updates without the client constantly asking the server for changes.
 - This is handled using WebSockets, which keep a persistent connection open between client and server.
 - So instead of polling, the server can push updates directly:
 - New chat messages
 - Reservation status changes
 - Payment confirmations
 - Time expires
 - Event-Driven Messaging (Inbound Webhooks)
 - Payments are handled in an event-driven way
 - Instead of the system constantly checking (polling) Ozow, Ozow sends a webhook to the backend when a payment completes.
 - That webhook triggers the next steps, like PIN generation and delivery to the buyer.
 - So, the webhook is the source of truth for payment confirmation. The redirect from Ozow is just for user experience and doesn't count as confirmation.
- 3) Architectural Patterns per Subsystem
- Layered (n-Tier) Architecture – System Wide
 - UniTrade follows a layered architecture with five clear layers:
 - Presentation Layer – React web app and React Native app. Handles UI and input only.
 - Edge Layer – Reverse proxy/CDN layer. Handles TLS, routing, and protection.
 - API Layer – Receives requests, validates input, enforces authorization, and routes traffic. No business logic here.
 - Application Layer – Core business logic, organised into modules like identity, Listings, Reservations, Payments, Chat, etc.
 - Data Layer – Database, cache, blob storage, and search systems.
 - Modular Monolith – Application Layer
 - Inside the application layer, UniTrade uses a modular monolith instead of microservices.
 - All business domains live in a single deployable system, but they are separated into modules like:
 - Identity
 - Listings
 - Reservations
 - Payments
 - Chat
 - Disputes
 - Notifications
 - Audit
 - This approach was chosen because the domains are tightly connected. For example, creating a reservation touches multiple modules at once (Listings, Reservation, Notifications, Audit).
 - Splitting this into microservices would introduce:
 - Distributed transaction complexity
 - Extra network latency
 - More operational overhead
 - So, the modular monolith keeps things simpler while enforcing boundaries between domains. It can still be split later if needed.
 - Domain-Driven Design – Module Boundaries

- The modules are structured using Domain-Driven Design (DDD) principles.
- Each module represents a bounded context:
 - Identity handles users and verification
 - Listings manages listing lifecycle
 - Reservations handles booking logic and timing
 - Payments manages transactions and PIN flow
- Modules don't directly access each other's internal data. They communicate through defined interfaces or events. This avoids tight coupling and keeps the system easier to evolve.
- State Machine Pattern – Reservations
 - Reservations are handled using a state machine, since the flow has multiple stages and rules.
 - A reservation moves through states like:
 - Created
 - Seller acknowledged
 - Coordinating
 - Completed
 - Expired
 - Cancelled
 - Each transition is controlled by either user actions or system events (like timer expiry or PIN entry).
 - This avoids scattered logic across the system. Without a state machine, you'd end up with fragmented rules in different places (timers, controllers, handlers) which becomes hard to reason about.
 - With a state machine:
 - Invalid transitions are impossible
 - All rules are centralised
 - State changes are predictable
 - Every transition can be logged cleanly in the Audit system.
- Event-Driven Pattern – Async Processing Layer
 - The async layer (Service Bus + workers) follows an event-driven architecture
 - The application publishes events like:
 - Reservation created
 - Payment confirmed
 - Listing submitted for review

Design Patterns

- 1) Strategy Pattern
 - Used for AI-based decision routing. The system dynamically chooses actions (auto-approve, reduce visibility, escalate to admin, etc.) based on risk/confidence scores.
- 2) State Pattern
 - Used in listing and use verification lifecycles. Listings and users move through defined states, and allowed actions depend on the current state.
- 3) Chain of Responsibility
 - Used in the listing submission pipeline. Each step processes the listing in sequence (verification, AI scoring, moderation, notifications) before passing it along.
- 4) Observer Pattern
 - Used for notifications. The system reacts to events such as state changes, meetup reminders, check-ins, and reservation expires by notifying users automatically.
- 5) Template Method Pattern
 - Used in dispute handling. Different dispute types follow the same overall workflow, but each dispute applies its own specific evaluating logic.
- 6) Command Pattern
 - Used for admin moderation actions like approve, reject, or request changes. Each action is treated as a discrete command with logging and downstream effects.
- 7) Decorator Pattern
 - Used for listing visibility modifiers. Medium-risk listings gain additional behaviours such as reduced visibility without changing the core listing object.
- 8) Saga Pattern
 - Used in reservation timer logic. The reservation process is a long-running, multi-step workflow with timeouts, branching, and rollback behaviour such as auto-expiry.

Constraints

- 1) **Ozow Payment Dependency** – The system must use Ozow as the only payment gateway. The architecture must support public webhooks, separate redirect handling from transaction verification, and avoid storing payment credentials
- 2) **University Email Validation** – Only verified South African university students may register. Email domains must be validated against a maintainable list of recognised SA university domains.
- 3) **PDF-Only Proof of Registration** – The upload system only accepts PDF proof-of-registration documents because the AI verification pipeline is designed specifically for PDFs.
- 4) **AI as Advisory Only** – AI systems may assist with scoring and verification but cannot make final rejection decisions independently. Medium- and high-risk cases must allow admin review.
- 5) **Admin Review SLA** – The system must support a visible and auditable moderation queue so admins can resolve flagged cases within the required 2–3 day SLA.
- 6) **Optional Location Permissions** – Meetup verification can use location data, but users cannot be forced to grant location access. The architecture must handle both verified and unverified check-ins.
- 7) **No Refund Support** – Refunds are intentionally excluded from the platform. Payment states are forward-only, with disputes handled separately rather than through transaction reversals.
- 8) **Mobile-First Constraint** – Since university students mainly use mobile devices and limited data connections, the frontend must prioritise responsive, low-bandwidth operation.
- 9) **Data Minimisation & POPIA Compliance** – The system may store only essential user and transaction data. Sensitive payment details cannot be stored, and personal information must be securely protected in line with Protection of Personal Information Act (POPIA).
- 10) **Evolving External Dependencies** – Constraints may need revision if Ozow's APIs, POPIA requirements, or the recognised university domain list changes.

Technology Requirements

Frontend (Web): React.js

We chose React for Unitrade because it directly supports the project's core goals of scalability, usability, and efficient development. React enables a unified development across web and mobile through its alignment with React Native. This alignment allows the team to build the web and mobile app using the same technologies, patterns and logic, thereby reducing development time, and ensuring consistent behavior across platforms. React's component-based architecture design is ideal for building interactive, modular interfaces such as product listings, carts, messaging systems, and notifications. This setup lets us develop, test, and update features independently. It also makes it easier to respond quickly to user feedback and keeps improving usability. Overall, React supports rapid development, maintainability, and a consistent user experience. This makes React a practical choice for a scalable and user-friendly marketplace like Unitrade.

Mobile: React Native

We chose React Native because it is more efficient, has better ecosystem support and lets us work faster. This choice fits Unitrade's goals for scalability and usability. React Native allows reuse of business logic, API handling, and data models between the mobile app and the React.js web app using TypeScript. This reduces the risk of duplication and ensures consistent behaviour. React Native has more mature and widely tested libraries for key features like maps, payments, notifications, and image uploads. Overall, React Native improves development speed, maintainability, and user experience, making it a better choice for a fast-evolving student marketplace like Unitrade.

Backend API: ASP.NET CORE (C#)

We chose ASP.NET Core (C#) because it provides a secure, scalable, and cloud-ready backend aligned with Unitrade's requirements. The framework is fully compatible with Azure, this simplifies deployment, scaling, and system monitoring. Built-in security features like ASP.NET Identity and JWT authentication provide high levels of data protection, ensuring POPIA compliance with minimal custom code. Overall, ASP.NET Core delivers reliable security, scalability, and efficient development for a growing student marketplace.

Primary Database: MSSQL + Redis

We chose MSSQL as our primary database because our data is highly structured and relational, requiring strong consistency, enforced relationships to meet POPIA requirements. MSSQL enhances security and reliability through encryption, backup, and scalability. Redis is used as a high-speed cache layer to improve efficiency. The cache provides quick access to commonly accessed information, such as shopping carts and sessions, as well as real-time capabilities, including token revocation, rate limiting, and payment security.

AI Verification and Material Suggestions

We are using AI to ensure trust and enhance efficiency in student transactions. Image verification: Azure Computer Vision automatically checks listing images for correct items, condition, and text. At delivery, it verifies the received product item against the listing to prevent fraud. This ensures buyers get what they ordered and protects payments. Material suggestions: The OpenAI API generates degree-specific materials lists and finds the lowest available prices. In addition, it helps students plan purchases by optimizing their budgets for

entire degrees. Overall, AI enhances reliability, usability, and efficiency in a student-focused marketplace.

Hosting: Azure App Services + Azure Container Apps

We are choosing to host Unitrade on Microsoft Azure to provide a scalable, reliable, and low maintenance platform. Azure App Services: For the MVP and early growth, App Services helps deploy applications without worrying about infrastructure, scales automatically during high loads, and allows focusing solely on developing the product. Azure Container Apps: With increasing users, container-based services (API, chat, notification, and AI) can independently scale to address burst loads without dealing with Kubernetes complexity. CI/CD Integration: GitHub Actions automates testing, security scanning, and deployments with blue-green deployments ensuring safe, continuous updates.

Payments: Ozow

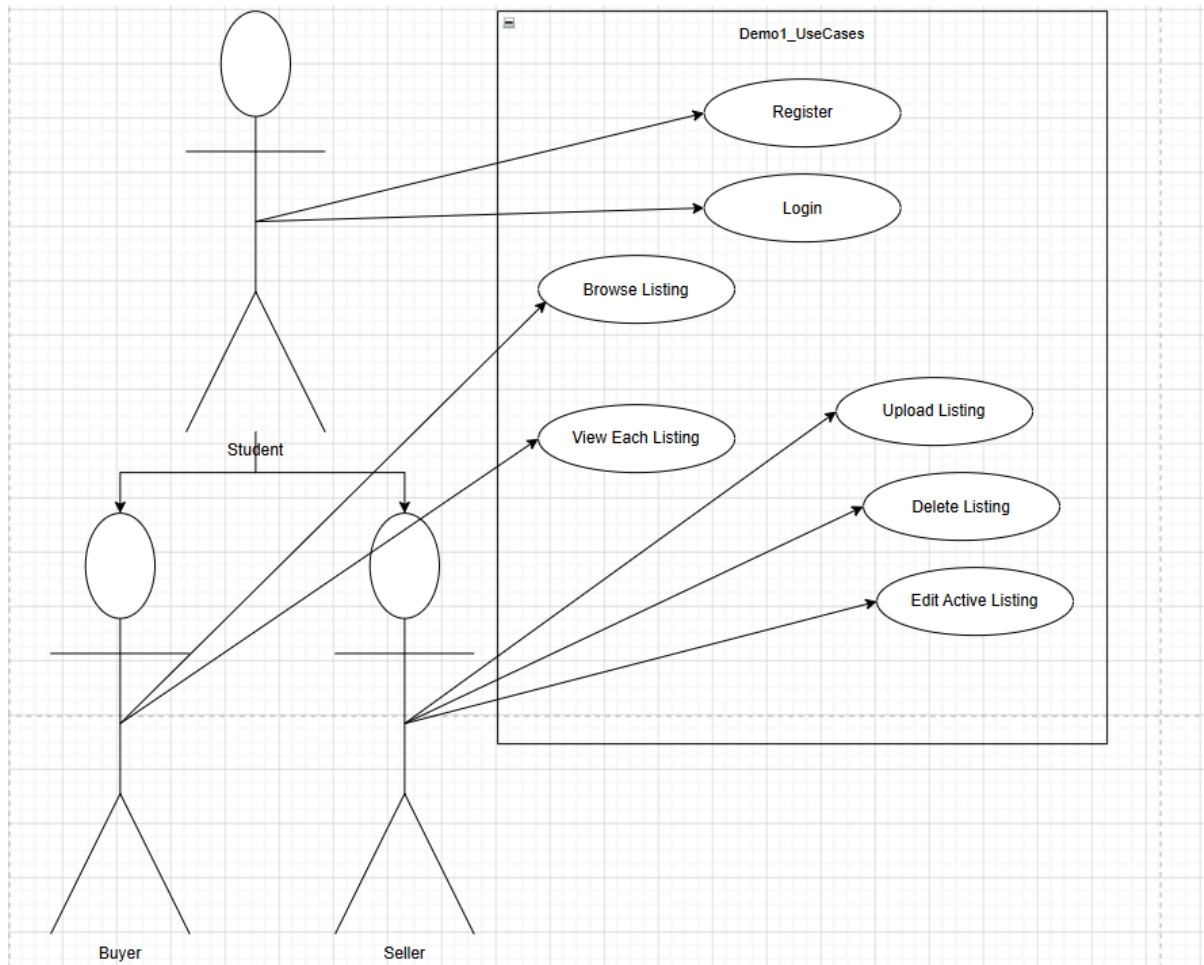
We chose OZOW for our payments processing for the following reasons. Payers make payments from their bank accounts without using cards, which is very useful in case you want to pay on delivery, and you forgot your card. OZOW is also compatible with all major SA banks. OZOW also does not have any monthly charges, only per-transaction fees. We chose OZOW for our payments processing for the following reasons. Payers make payments from their bank accounts without using cards, which is very useful in case you want to pay on delivery, and you forgot your card. OZOW is also compatible with all major SA banks. OZOW also does not have any monthly charges, only per-transaction fees.

Notifications: FCM + Resend

We chose to use a multi-channel notification system to ensure students are informed instantly and reliably FCM: Instant push notifications for our app. Resend: Transactional emails for sign-ups, password resets and invoices.

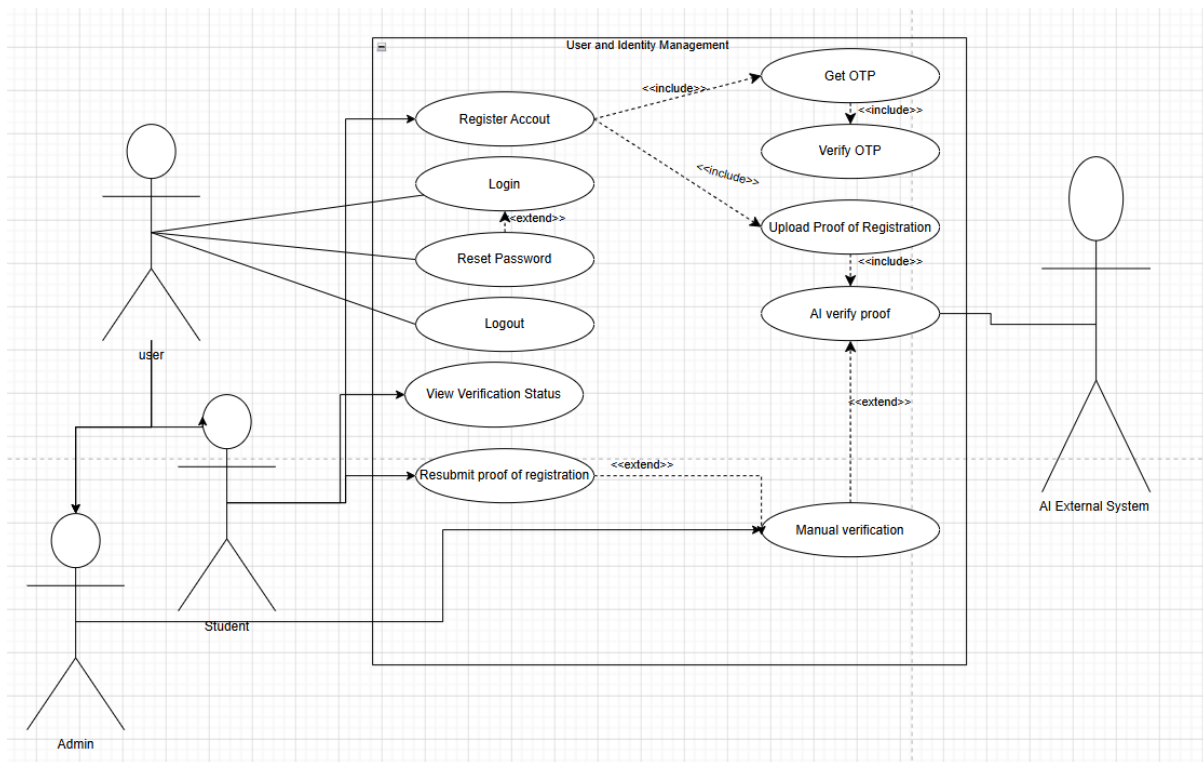
4. Use Case Diagrams

Demo_1 Use Cases

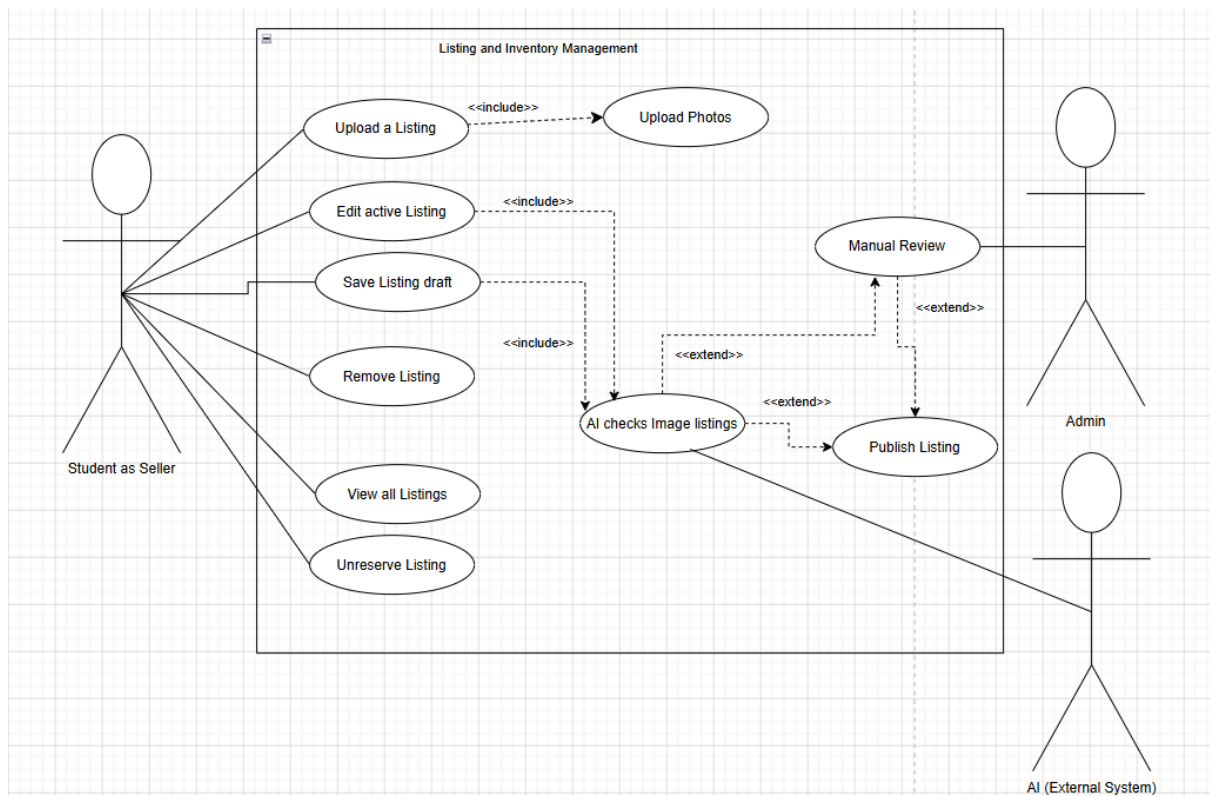


Overall System Use Cases

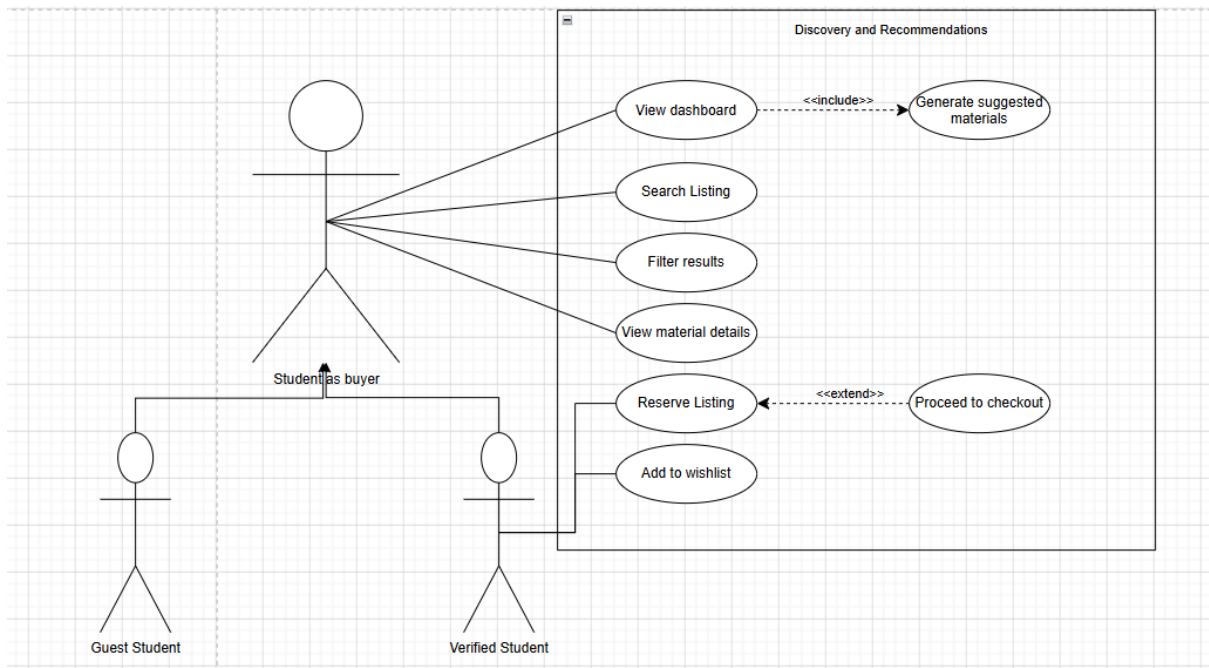
4.1 User and Identity Management



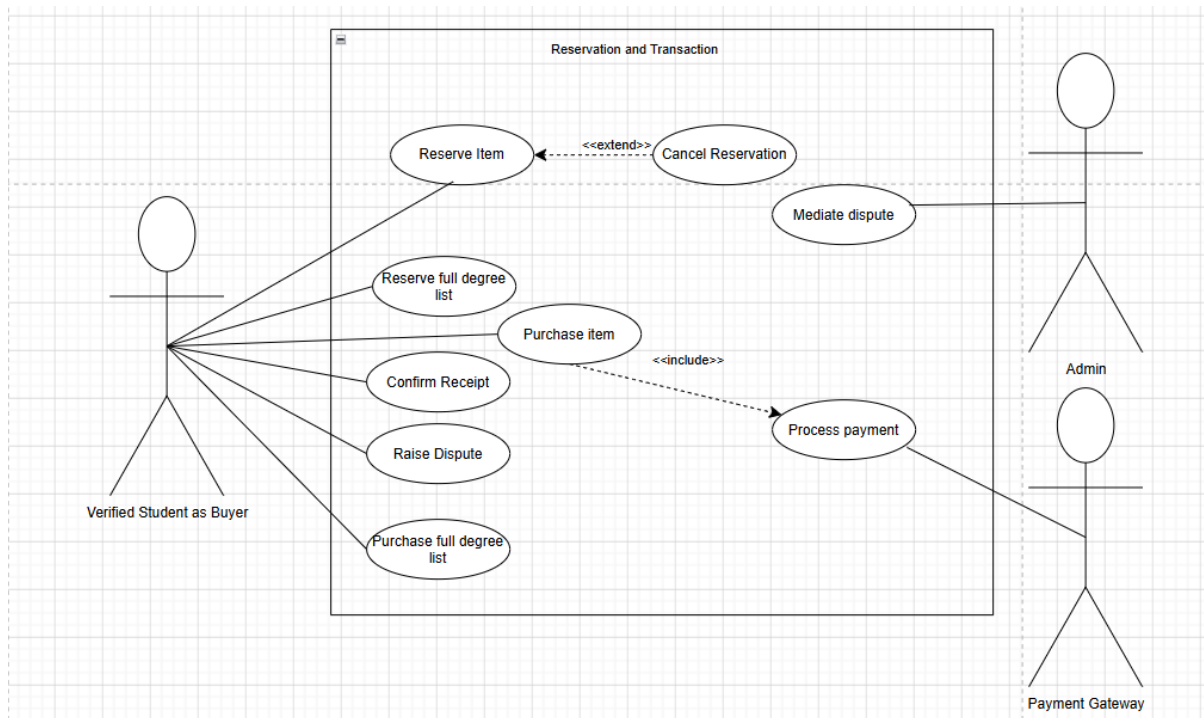
4.2 Listing and Inventory Management



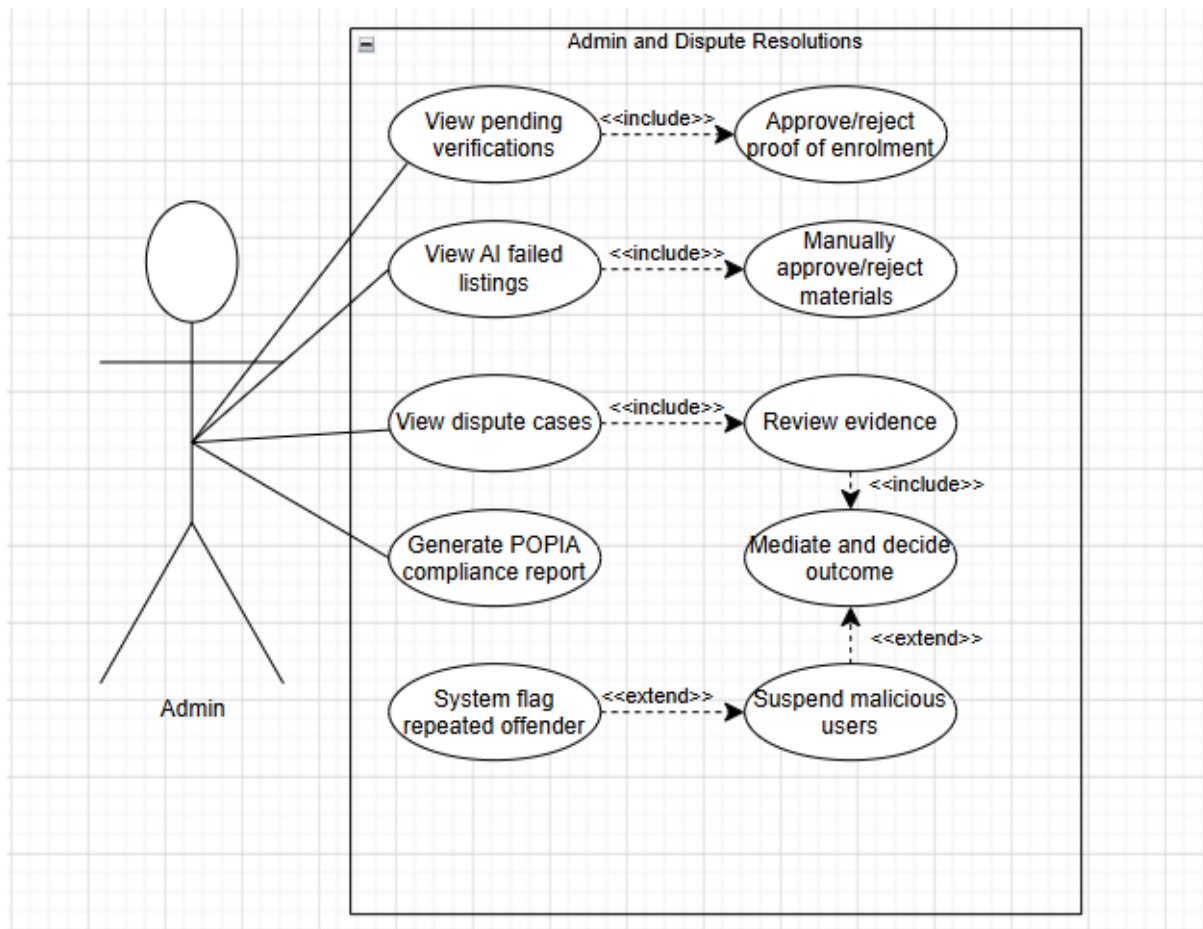
4.3 Discovery and Recommendations



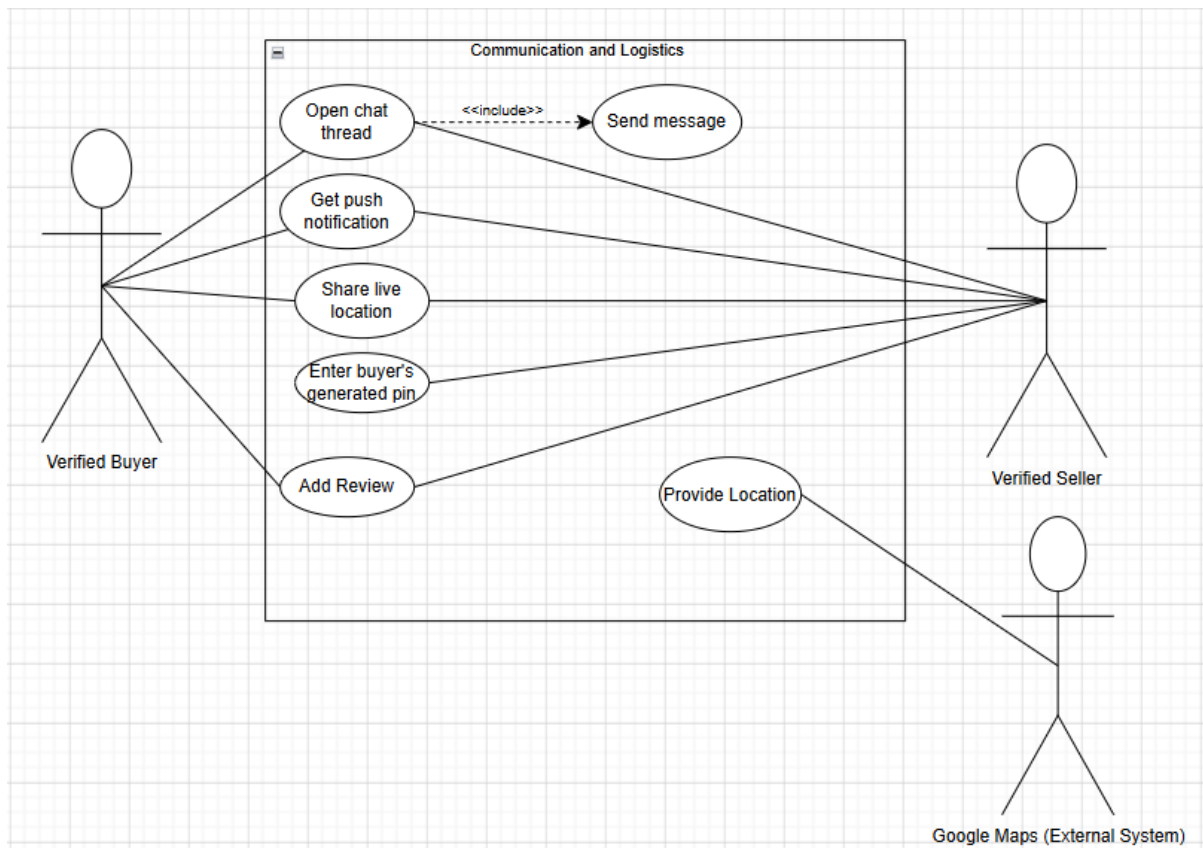
4.4 Reservation and Transaction



4.5 Admin and Dispute Resolution



4.6 Communication and Logistics



5. Epics, User Stories and Acceptance Criteria

Epic 1: User and Identity Management

Enable South African university students to register, authenticate, and verify their student status so that the marketplace remains exclusive to legitimate students.

US-1.1 — Register an Account

As a: prospective student user I want to register with my name, surname, university, degree, year of study, student email, and password so that I can begin the process of accessing the marketplace.

Acceptance Criteria:

- Given I am on the registration screen, when I submit valid details with a recognised SA university email domain, then my account is created in pending verification state.
- Given I submit an email with a non-SA-university domain, when I attempt to register, then registration is rejected with a clear message naming the domain check as the reason.
- Given I submit an email already registered, when I attempt to register, then I see a message prompting me to log in or reset my password (the system does not reveal whether the account is verified).
- Given I am on the registration screen, when I submit the form with missing or invalid required fields, then I see field-level error messages and the form is not submitted.

US-1.2 — Receive and Verify OTP

As a: newly registering user I want to receive a one-time PIN on my student email and enter it so that the system confirms I control the email address.

Acceptance Criteria:

- Given my account was just created, when registration completes, then an OTP is sent to my student email within 60 seconds.
- Given I enter the correct OTP within its validity window, when I submit, then my email is marked verified and I proceed to the proof-of-registration step.
- Given I enter an incorrect OTP, when I submit, then I see an error and remain on the OTP screen; after a configured number of failed attempts, the OTP is invalidated and I must request a new one.
- Given the OTP has expired, when I submit it, then the system rejects it and offers to resend.
- Given I request a resend, when triggered, then a new OTP is sent and the previous one is invalidated; resends are rate-limited.

US-1.3 — Upload Proof of Registration

As a: verified-email user I want to upload my proof of registration as a PDF so that the system can verify I am a current student.

Acceptance Criteria:

- Given I have verified my email, when I reach the upload step, then I can select a PDF file from my device.
- Given I attempt to upload a non-PDF file, when I submit, then the upload is rejected with a message stating PDF is required.
- Given the file exceeds the maximum size, when I submit, then the upload is rejected with a clear size limit message.
- Given a successful upload, when complete, then my proof is queued for AI verification and my status updates to 'under review'.

US-1.4 — AI-Assisted Proof Verification (System-Driven)

As the: system I want to automatically score uploaded proofs of registration for legitimacy so that confident-positive cases are auto-approved without admin workload.

Acceptance Criteria:

- Given a PoR has been uploaded, when AI scoring runs, then it returns a confidence score and a routing decision: auto approve, manual review, or manual review high risk.
- Given the score is in the auto-approve band, when scoring completes, then the user's account is upgraded to verified and the user is notified in-app and by email.
- Given the score is in the manual-review band, when scoring completes, then the case is added to the admin verification queue and the user remains in 'under review'.
- Given the score is in the manual-review-high-risk band (clear forgery indicators), when scoring completes, then the case is also routed to the admin queue with a high risk flag — the AI never auto-rejects; rejection is always a human decision.
- Given a user has uploaded a PoR, when AI scoring is in progress, then the user sees a loading indicator until a result is returned.
- Given a PoR has been scored, when the result is stored, then the system records the score, decision, timestamp, and model version for audit purposes.

US-1.5 — Manual Verification by Admin

As an: admin I want to review proofs of registration that AI flagged for manual review so that legitimate students get verified and forgeries are rejected.

Acceptance Criteria:

- Given a case is in the admin verification queue, when I open it, then I see the uploaded PoR, the user's submitted details, the AI score, and any AI-flagged reasons.
- Given I approve the case, when I confirm, then the user's account is upgraded to verified and they receive an in-app and an email notification.
- Given I reject the case, when I confirm, then I must select a terminal rejection reason (forged, expired, wrong document type, other) and add a free-text note; the user is notified and cannot resubmit without admin intervention.

- Given I request changes, when I confirm, then I must select a request changes reason (unreadable, missing information, other) and add a free text note; the user is notified with the reason and prompted to resubmit.
- Given I perform any verification action (approve, reject, request changes), when the action is confirmed, then the system records the admin ID, action taken, timestamp, and reason for audit purposes.
- Given a case has exceeded the defined SLA threshold, when I view the verification queue, then the case is prioritised and surfaced at the top of the queue.

US-1.6 — Resubmit Proof of Registration

As a: user whose PoR was rejected or required changes I want to upload a new PoR so that I can complete verification.

Acceptance Criteria:

- Given my PoR is in the 'changes requested' state, when I log in, then I see a clear prompt to resubmit and the reason for the previous rejection.
- Given my PoR is in the 'changes requested' state, when I check my emails, then I see a clear reason why my submission was declined and an instruction to resubmit.
- Given my PoR has been rejected after a manual admin review due to legitimacy concerns (e.g., forgery), when I attempt to access the application, then I am denied access and I receive both an in-app rejection notice and an email explaining the reason for the decision.
- Given my PoR is in the 'changes requested' state and the request was made by Admin A, when I resubmit a new PoR, then the resubmission is assigned to Admin A for review (not a different admin).

US-1.7 — View Verification Status

As a: registered user I want to see my current verification state at any time so that I know what I can and can't do on the platform.

Acceptance Criteria:

- Given I am logged in, when I open my account/profile area, then I can see my current state: pending email verification, pending PoR upload, under review, verified, rejected, or changes requested.
- Given I am in 'under review', when I view my status, then I see an estimated time remaining (based on SLA) and a note that drafts can be created but listings won't go live until verified, and I can browse listings.
- Given I am in 'rejected' or 'changes requested', when I view my status, then I see the reason and any next-step actions.

US-1.8 — Login

As a: registered student I want to log in with my student email and password so that I can access my account.

Acceptance Criteria:

- Given I enter correct credentials, when I submit, then I am authenticated and routed to the appropriate landing page based on my verification status.
- Given I enter incorrect credentials, when I submit, then I see a generic 'invalid email or password' message.
- Given I am logged in and idle for longer than the configured inactivity timeout (e.g. 30 minutes), when I attempt to access a protected page, then my session is expired and I am redirected to the login page.
- Given I check the 'remember me' box during login, when I close and reopen my browser or return after a long period (but within the extended session lifetime), then I am still required to re-authenticate.

US-1.9 — Reset Password

As a: student who has forgotten my password I want to reset it via an email-verified flow so that I can regain access.

Acceptance Criteria:

- Given I select 'forgot password', when I enter my registered email, then a password reset OTP is sent to my email.
- Given I submit a valid OTP within its validity window, when I enter a new password, then my password is updated and I am logged in.
- Given the OTP has expired or been used, when I attempt to use it, then it is rejected and I am prompted to start the reset flow again.

US-1.10 — Logout

As a: logged-in student I want to log out so that my session ends and my account remains secure on shared devices.

Acceptance Criteria:

- Given I am logged in, when I select logout, then I am successfully logged out of the system.
- Given I have logged out, when I attempt to access any authenticated route, then I am redirected to the login screen.
- Given I have logged out, when I try to perform any authenticated action, then it is rejected as unauthorised.

Epic 2: Listing and Inventory Management

Verified students can create, manage, and remove listings. Every submitted listing is auto-scored by AI; high-risk listings route to admin review before going live. Unverified students can save drafts during the verification window — drafts are invisible to buyers and auto-submit once verification completes. Sellers see every listing's state at all times.

US-2.1 — Create a Listing Draft

As a: logged-in student I want to save my work-in-progress as a draft so that I don't lose it while composing or while I'm waiting for verification.

Acceptance Criteria:

- Given I am a logged-in student (verified or not) creating or editing a listing, when I choose to save as draft even with required fields missing, then the system saves the draft without showing validation errors for required fields.
- Given I am an unverified student, when I save a listing as a draft, then the draft is flagged with status 'pending verification'.
- Given I am an unverified student with one or more completed drafts, when my account becomes verified, then each flagged draft is automatically submitted for AI scoring without any manual action from me.
- Given I am the owner of a saved draft, when I open it from my drafts list, then I can edit any field and save it again as a draft.
- Given I am the owner of a saved draft, when I choose to delete it, then the draft is permanently removed and no longer appears in my drafts list.

US-2.2 — Submit Listing for Review

As a: verified student I want to submit a draft listing into the review pipeline so that buyers can discover and buy it.

Acceptance Criteria:

- Given I am a verified student, when I submit a draft and required fields are missing, then the system shows validation errors and does not enter AI scoring.
- Given I am a verified student and I submit a complete draft, when submission completes, then the listing enters AI scoring and I am notified of the resulting state: live, live with reduced visibility, or under review.
- Given I open an existing draft and submit it, when submission is successful, then the original draft is consumed by the submission.

US-2.3 — AI Score Listing (System)

As the: system I want to score every submitted listing for risk so that low-risk listings go live, medium-risk listings go live with reduced visibility, and high-risk listings route to admin review.

Acceptance Criteria:

- Given a submitted listing receives a low-risk score, when scoring completes, then the listing is published with full visibility.
- Given a submitted listing receives a medium-risk score, when scoring completes, then the listing is published with reduced visibility (e.g. lower search rank, hidden from recommendations) and queued for non-urgent admin review.
- Given a submitted listing receives a high-risk score, when scoring completes, then the listing is not published, is held in the admin review queue, and that queue is prioritised over medium-risk background reviews.
- Given a listing has been scored, when scoring finishes, then the system logs: timestamp, risk score, flagged reasons, and the model version used.

US-2.4 — Manual Review

As an: admin I want to review AI-flagged listings so that legitimate ones reach buyers and problematic ones don't.

Acceptance Criteria:

- Given a listing is in the admin review queue, when I open it, then I see the listing details, photos, AI score, AI-flagged reasons, and the seller's history.
- Given I approve a flagged listing, when I confirm, then the listing is published and the seller is notified.
- Given I reject a flagged listing, when I confirm, then I must select a reason and add a free-text note; the listing is removed from the platform and the seller is notified with the reason.
- Given I request changes on a flagged listing, when I confirm, then I must provide a reason; the listing is returned to the seller as a draft (not live) accompanied by the reason, and the seller can edit and resubmit.
- Given I make any decision (approve, reject, request changes), when I confirm, then the system logs: admin ID, timestamp, outcome, and the reason text provided.

US-2.5 — Edit Active Listing

As a: seller I want to edit one of my live listings so that I can correct or update it.

Acceptance Criteria:

- Given my listing is reserved (a buyer has an active reservation), when I attempt to edit it, then I can only modify non-material fields (typo fixes, description clarifications); price, photos, and condition are locked.
- Given I edit a live listing (whether reserved or not), when I save the edit, then the listing is re-scored by AI and may result in reduced visibility or admin review; existing reservations remain valid and are not cancelled.
- Given I save any edit, when the edit is applied, then the system logs the change (timestamp, old value, new value, my user ID).

US-2.6 — Remove Listing

As a: seller I want to remove a listing from circulation so that it no longer appears on the marketplace.

Acceptance Criteria:

- Given I am removing a listing, when the removal flow begins, then I am presented with a reason picker (sold off-platform, no longer for sale, other).
- Given my listing is currently reserved by a buyer, when I attempt to remove it, then I must confirm twice (checkbox + confirm button), and the buyer is automatically notified of the removal.

US-2.8 — View All Listings

As a: seller I want to see all my listings in one place so that I can manage my inventory.

Acceptance Criteria:

- Given I am viewing my listings, when I apply a state filter (draft, live, reduced visibility, under review, removed), then only listings in the selected state are shown.
- Given I am viewing my listings, then each listing displays its state, AI score (if available), admin notes (if any), edit history, and reservation history.
- Given a listing is in a non-obvious state (e.g. reduced visibility, under review), when I view it, then I see an explanation of the state and any next steps.
- Given I am viewing my listings, when I apply a sort, then listings are ordered by the chosen criterion (date, state, price, expressions of interest).

Epic 3: Discovery and Recommendations

Help verified students find the materials they need — through casual browsing, intentful search, behaviour-driven recommendations, and a degree-aware planner that anticipates what they'll need across their studies. Surface seller reputation alongside listings so buyers can make informed decisions before expressing interest.

US-3.1 — Browse Listings

As a: student I want to browse the marketplace without a specific query so that I can casually explore what's available.

Acceptance Criteria:

- Given I access the marketplace without a search query, then I see a paginated or scrollable list of active listings.
- Given I am viewing browse results, then listings are sorted by relevance to my profile.
- Given I am viewing browse results, then each listing displays title, price, condition, primary photo, seller name, and seller rating summary.
- Given I am viewing browse results, when I apply filters (category, price range, condition, course/module, university) or sorting (newest, price asc/desc, relevance), then results update accordingly.
- Given I am an unverified student, when I browse listings, then I can view listings and I see a banner indicating limited access.

Epic 4: Reservation and Transaction

Enable verified students to reserve listings, coordinate and complete in-person meetup transactions, and confirm completion through a PIN-based closure system. Payment is processed at the point of physical inspection via Ozow. No refunds are issued — physical inspection before payment is the buyer's protection.

US-4.1 — Reserve a single item

As a: verified buyer I want to reserve a listing I'm interested in so that it is held for me while I arrange the meetup with the seller.

Acceptance Criteria:

- Given I am a verified student viewing an active, unreserved listing, when I express interest, then a reservation is created, the listing is marked as reserved, and the seller receives an automated initial message on my behalf.
- Given a reservation has been created, when it is active, then the listing is no longer available for other buyers to reserve.
- Given a reservation exists, when neither party has progressed, then the system automatically cancels the reservation and the listing returns to active circulation. • Given a reservation is auto-cancelled, when the cancellation fires, then both the buyer and seller are notified in-app.
- Given I am an unverified student, when I attempt to reserve a listing, then the action is blocked and I am shown a message explaining that verification is required.

US-4.2 – Reserve seller bundle from degree list

As a: verified buyer, I want to reserve all relevant listings from a single seller that match my degree list so that I can arrange a bulk purchase in one meetup.

Acceptance Criteria:

- Given I confirm a bundle from a specific seller, when I proceed with reservation, then a reservation is created that includes all selected, available listings from that seller.
- Given some listings from that seller are already reserved, when I create the bundle reservation, then only unreserved listings are included and I am informed which items were unavailable.
- Given a bundle reservation is created, then the seller is notified that this is a bundle purchase request and the reservation is treated as a single grouped transaction.
- Given a bundle reservation is active, then it follows a 7-day reservation window instead of the 24-hour reservation logic.
- Given a bundle reservation exists, then all items are grouped under one reservation context and cannot be partially confirmed through separate flows.

US-4.3 – Cancel reservation (buyer-initiated)

As a: buyer I want to cancel a reservation I created so that the seller's listing returns to circulation if I am no longer interested.

Acceptance Criteria:

- Given I have an active reservation, when I chose to cancel it, then the reservation is removed, the listing returns to active, and the seller is notified.
- Given I cancel a reservation, when the cancellation is confirmed, then the system records the cancellation timestamp and the initiating party for reputation tracking purposes.

US-4.4 – Release reservation (seller-initiated)

As a: seller I want to release a reservation when the buyer is unresponsive so that my listing returns to active circulation.

Acceptance Criteria:

- Given a reservation exists on my listing, when I attempt to release it, then the release is only permitted if the buyer has been inactive for at least 12 hours since the reservation was created.
- Given the 12-hour cooldown has not elapsed, when I view the reservation, then the release option is disabled and I see a message indicating how much time remains before I can release.
- Given I release a reservation, when confirmed, then the listing returns to active, the buyer is notified, and the event is logged.

US-4.5 – Purchase single item

As a: buyer with an active reservation I want to pay for an item at the meetup after inspecting it so that the transaction is completed only when I am satisfied.

Acceptance Criteria:

- Given I have met the seller and inspected the item, when I choose to proceed with payment, then I am directed to the Ozow payment flow with the correct amount pre-populated.
- Given payment via Ozow is successful, when the payment is confirmed, then the system generates a transaction PIN and delivers it to me.
- Given payment fails, when the Ozow flow returns a failure, then I am shown a clear error message and can retry without the reservation being cancelled.
- Given payment is successful, when the transaction is recorded, then the system stores: transaction ID, buyer ID, seller ID, item ID, amount, and timestamp — no card or bank details are stored.

US-4.6 – Purchase full degree list

As a: buyer with reservations across my degree list I want to process payment for all reserved items in one flow so that I can complete a bulk transaction at a single meetup.

Acceptance Criteria:

- Given I have active reservations for multiple listings, when I choose to purchase the full degree list, then I am taken to a payment flow that aggregates the total across all reserved items.
- Given payment is successful, when the flow completes, then a transaction PIN is generated per item and the seller is notified for each.

- Given some items in the list are no longer available (e.g. reservation expired before payment), when I initiate the bulk purchase, then those items are excluded and I am informed before confirming payment.

US-4.7 – Confirm receipt (PIN verification)

As a: seller I want to enter the PIN shown to the buyer after payment so that the transaction is officially closed once I confirm the handover happened.

Acceptance Criteria:

- Given the buyer has completed payment, when the Ozow flow succeeds , then a PIN is generated and displayed to the buyer in-app.
- Given the buyer has shown me their PIN, when I enter it correctly in-app, then the transaction is marked complete and both parties are prompted to leave a review.
- Given I enter an incorrect PIN, when I submit, then I see an error and can retry; after a configured number of failed attempts the PIN is invalidated and the case is flagged for support.
- Given the transaction is completed via PIN, when confirmed, then the systems logs the completion timestamp, the confirming party (seller), and triggers the post-transaction review prompt for both parties.

US-4.8 – System auto-cancel reservation

As the: system I want automatically cancel the stale reservations so that listings do not remain locked when buyers or sellers are unresponsive.

Acceptance Criteria:

- Given a reservation has been active for 24 hours with no meaningful progression (no seller reply to the automated message, or no buyer response after seller reply), when the threshold is crossed, then the reservation is automatically cancelled, and the listing returns to active.
- Given an auto-cancellation fires, when it is executed, then both parties receive an in-app notification, and the event is logged with a reason of auto-expired.

US-4.9 – View reservation and transaction history

As a: student (buyer or seller) I want to see all my past and active reservations and completed transactions so that I can track my activity on the platform.

Acceptance Criteria:

- Given I am a verified student, when I navigate to my reservation/transaction history, then I see a list of all active reservations and completed transactions, with state, item, counterparty name, and date.
- Given I select a specific transaction, when I open it, then I see the full details: item, amount, meetup outcome, PIN confirmation status, and any associated review.
- Given I have an active reservation, when I view it, then I can see the time remaining before auto-cancellation and any relevant next steps.

Epic 5: Disputes and Moderation

Provide the channels through which users raise concerns about the marketplace and admins act on those concerns or on system-detected patterns. Disputes are limited to what the platform can reliably arbitrate — pre-payment listing quality complaints — and remedies are reputational, not financial. Automatic reputation flags can be challenged through a lightweight mechanism. Admin actions can be appealed through a structured process with a different reviewer, time limits, and bounded outcomes.

US-5.1 – File listing complaint

As a: buyer who attended a meetup, I want to file a complaint about a listing so that misleading or inaccurate listings can be acted on.

Acceptance Criteria:

- Given I attended a meetup for a listing, when I attempt to file a complaint, then it is accepted only within the configured time window after the meetup, and submissions outside this window are rejected.
- Given I am filing a complaint, then I must provide either: photos taken in-app at the meetup, or a refusal flag indicating the seller declined photos —and I cannot submit the complaint without one of these.
- Given I provide photos as evidence, then photos must be captured using the in-app camera, each photo includes a timestamp, and gallery uploads are not allowed.
- Optional: buyer may include a written description of the discrepancy.
- Given a complaint is successfully submitted, then it is routed to the admin dispute queue and includes reservation details, listing, evidence (photos/refusal flag), and chat history.
- Given a complaint has been reviewed, then the buyer is notified of the outcome.

US-5.2 – Dispute reputation flag

As a: user who has received a reputation flag, I want to dispute it so that incorrect or unfair flags can be reviewed.

Acceptance Criteria:

- Given I have received an automatic reputation flag, when I view my account status, then I can dispute the flag within the configured time window, and the option is not available after the window expires.
- Given I choose to dispute a flag, then I must provide a brief written explanation, and I can submit without additional evidence.
- Given a dispute is submitted, then it is queued for admin review, and the admin either removes the flag or lets it stand, and the decision is final.
- Given I submit disputes excessively within a defined period, then my access to the dispute mechanism is temporarily suspended.

US-5.3 – Appeal decision

As a: student who received an admin-issued action (strike, suspension, removal) I want to appeal the decision so that I have recourse when an admin has misread evidence or new information has come to light.

Acceptance Criteria:

- Given I received an admin-issued action, when I attempt to appeal, then I can submit an appeal within 7 days of the action, I cannot appeal the same action more than once, and appeals outside this window are not allowed.
- Given I am submitting an appeal, then I must provide either new evidence or a valid explanation of how the original decision misread existing evidence, and appeals based on disagreement alone are rejected.
- Given an appeal is submitted, then it is assigned to a different admin than the original decision-maker.
- Given an appeal is reviewed, then the outcome is one of: uphold, reduce penalty, or overturn, and the penalty cannot be increased.
- Given an appeal decision is made, then the user is notified with reasoning and the decision is final.

US-5.4 – Resolve dispute

As an: admin I want to review and resolve filed disputes so that legitimate complaints result in seller strikes and frivolous ones are dismissed.

Acceptance Criteria:

- Given I am viewing the dispute queue, then disputes are ordered by age (oldest first) and time-sensitive disputes are prioritised.
- Given I open a dispute, then I see the original listing, buyer evidence (photos or refusal flag), any description, chat history, and both users' reputation history.
- Given I review a dispute, then I can choose to uphold, dismiss, or request more information; and if upheld, the seller receives a strike and the listing is removed, and the buyer is notified of the outcome.
- Given I request more information, then the buyer is prompted to provide clarification and the dispute remains open until resolved.
- Given a buyer repeatedly submits dismissed disputes within a defined period, then this contributes to a buyer-side reputation flag.
- Given a dispute is resolved, then the decision, reasoning, admin ID, and timestamp are recorded, and decisions follow the defined standard of proof.

US-5.5 – Review flagged users

As an: admin I want to see users surfaced by automatic pattern detection so that I can decide whether to take action on cumulative behaviour that no individual incident warranted.

Acceptance Criteria:

- Given I am viewing the flagged-users dashboard, then I see users surfaced by automatic pattern detection when thresholds are met.
- Given I open a flagged user entry, then I see the user, triggered pattern type, underlying events with timestamps and source links, and full account history.
- Given I am viewing the dashboard, when I apply filters, then I can filter by pattern type and severity, and users with multiple overlapping patterns are surfaced higher.

US-5.6 - Take action on user

As an: admin I want to apply a consequence to a user based on a dispute outcome or flag pattern so that the marketplace is protected from bad actors.

Acceptance Criteria:

- Given I am reviewing a user case (dispute outcome or flag pattern), when I choose to take action, then I can apply one of: warning, strike, temporary suspension, or permanent removal.
- Given I am applying an action, then I must select a reason and provide a free-text note.
- Given an action is applied to a user, then the user is notified via in-app and email, and the notification includes the action type, reason, and duration where applicable.
- Given a user receives an action, then it is appealable according to appeal rules, and permanent removals due to fraud or POPIA-mandated takedowns are not appealable.
- Given an action is taken, then it is logged with admin ID, timestamp, user, action type, reason, and source case.
- Given a user accumulates strikes, then threshold rules trigger escalation conditions (e.g. review for suspension).

US-5.7 – Review appeal

As an: admin (different from the one who made the original decision) I want to review an appealed decision so that users have meaningful recourse against admin error.

Acceptance Criteria:

- Given I am viewing the appeal queue, then appeals are ordered by age (oldest first).
- Given I open an appeal, then I can see the original action, original admin reasoning, user appeal grounds, and all case evidence.
- Given I am assigned to review an appeal, then I cannot be the same admin who issued the original decision.
- Given I complete an appeal review, then I must provide written reasoning visible to the user, and the user is notified of the outcome.

Epic 6: Meetup Logistics

Provide students with map-based support for meetups arranged in chat, helping them find the agreed location and navigate there. Map functionality is provided through Google Maps integration.

Out of scope: chat, notifications, presence verification, location sharing during transit (privacy-preserving check-in is handled in package 4.)

US-6.1 – View meetup location on map

As a: student with an arranged meetup I want to see the agreed meetup location on a map so that I know where to go.

Acceptance Criteria:

- Given a meetup location has been agreed and captured in chat, then it is available in the related reservation/transaction view.
- Given I open a reservation with a meetup location, then I can view the location on an interactive in-app map and the meetup point is clearly marked.
- Given I am viewing the meetup map, then I can see surrounding landmarks, and I can see my current location for orientation.

US-6.2 – Pre-meetup attendance check-in

As a: student with an upcoming meetup, I want to confirm my attendance before the meetup so that the system can verify participation for audit purposes.

Acceptance Criteria:

- Given a meetup is scheduled, when it is 15 minutes before the meetup time, then both participants receive a check-in prompt.
- Given I receive a check-in prompt, then I can either confirm attendance or decline participation.
- Given I confirm attendance, then my confirmation is recorded for audit purposes.
- Given I decline attendance, then my participation is marked as forfeited according to the system rules.
- Given no response is given before the meetup time, then the system records the attendance as unconfirmed.